

PRACTICAL 10

Aim

To perform scalable customer segmentation using Apache Spark MLlib by implementing Parallel K-Means clustering on large-scale customer transaction data, applying feature extraction and normalization, evaluating clustering quality using WSSSE, selecting the suitable number of clusters using the Elbow Method, and interpreting customer segments.

Problem Definition

Customer transaction platforms generate a large amount of data containing information about customer spending, purchase frequency, average basket size, and purchase recency. Traditional single-node K-Means implementations may become inefficient or run into memory limitations when processing millions of customer records.

Apache Spark provides distributed data processing and Spark MLlib provides scalable machine learning algorithms that can process large datasets using parallel computation.

The task is to load and preprocess a large-scale customer transaction dataset using PySpark, extract relevant customer behaviour features, normalize the features, and apply Spark MLlib K-Means clustering.

The practical includes creating a SparkSession, loading the customer transaction dataset, examining and validating the data, extracting customer behaviour features, applying StandardScaler, partitioning the data, implementing K-Means with $k=5$, calculating WSSSE, evaluating the clusters, visualizing cluster characteristics, and interpreting the customer segments.

Objectives

- Understand scalable machine learning using Apache Spark MLlib.
- Create and configure a SparkSession.
- Load a large-scale customer transaction dataset.
- Examine the schema and records of the dataset.
- Validate the dataset for missing and incorrect values.
- Extract customer behaviour features.
- Use total spend, purchase frequency, average basket size, and purchase recency as clustering features.
- Apply feature scaling using StandardScaler.
- Understand the effect of Spark data partitioning.
- Implement Parallel K-Means using Spark MLlib.
- Perform customer segmentation using $k=5$.
- Calculate WSSSE for evaluating clustering quality.

- Apply the Elbow Method to select a suitable number of clusters.
- Visualize cluster centers and customer segments.
- Interpret the behaviour represented by each cluster.
- Compare Spark MLlib K-Means with single-node Scikit-learn K-Means.
- Understand the advantages of distributed machine learning for large datasets.

Output

The first screenshot shows the initial setup of a PySpark session and the generation of a simulated customer transactions dataset. The code includes imports for Spark, pandas, and various MLlib classes, followed by the creation of a SparkSession and the generation of 200,000 records based on predefined profiles.

```
[1]: import os
import sys
import time
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Set visualization styles
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['figure.dpi'] = 110

# Initialize PySpark Session
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, rand, expr, when, round as spark_round, count, avg, stddev
from pyspark.ml.feature import VectorAssembler, StandardScaler, StringIndexer, OneHotEncoder
from pyspark.ml.clustering import KMeans, BisectingKMeans
from pyspark.ml.evaluation import ClusteringEvaluator
import sklearn.cluster

spark = SparkSession.builder \
    .appName("Practical10_Scalable_KMeans") \
    .master("local[*]") \
    .config("spark.driver.memory", "4g") \
    .config("spark.sql.shuffle.partitions", "8") \
    .getOrCreate()

print(f"PySpark Session Successfully Started! Spark Version: {spark.version}")

PySpark Session Successfully Started! Spark Version: 4.2.0

[2]: # Generate Simulated Customer Transactions Dataset
np.random.seed(42)
num_records = 200000

profiles = [
    {"name": "High-Value Loyal", "weight": 0.15, "spend": (3500, 800), "freq": (40, 10), "basket": (120, 20), "recency": (8, 5)},
    {"name": "Recent Churn Risk", "weight": 0.20, "spend": (1800, 500), "freq": (25, 8), "basket": (75, 20), "recency": (110, 20)},
    {"name": "Casual Low-Spenders", "weight": 0.35, "spend": (250, 100), "freq": (4, 2), "basket": (20, 8), "recency": (60, 30)},
    {"name": "Frequent Mid-Spenders", "weight": 0.20, "spend": (1200, 300), "freq": (30, 6), "basket": (40, 12), "recency": (15, 8)},
    {"name": "VIP Power Buyers", "weight": 0.10, "spend": (10000, 1500), "freq": (70, 15), "basket": (250, 40), "recency": (5, 3)}
]
```

The second screenshot shows the continuation of the notebook, where the generated data is processed into a DataFrame, saved to CSV, and then loaded back into Spark for further analysis. The code also includes a preview of the first few rows of the generated data.

```
categories = ["Electronics", "Fashion", "Grocery", "Home & Garden", "Beauty"]

data_rows = []
for i in range(num_records):
    cust_id = 1000000 + i
    prof_idx = np.random.choice(5, p=[p["weight"] for p in profiles])
    p = profiles[prof_idx]

    spend = max(10.0, float(np.random.normal(p["spend"][0], p["spend"][1])))
    freq = max(1, int(np.random.normal(p["freq"][0], p["freq"][1])))
    basket = max(5.0, float(np.random.normal(p["basket"][0], p["basket"][1])))
    recency = max(1, int(np.random.normal(p["recency"][0], p["recency"][1])))
    category = np.random.choice(categories, p=[p["weight"] for p in profiles])

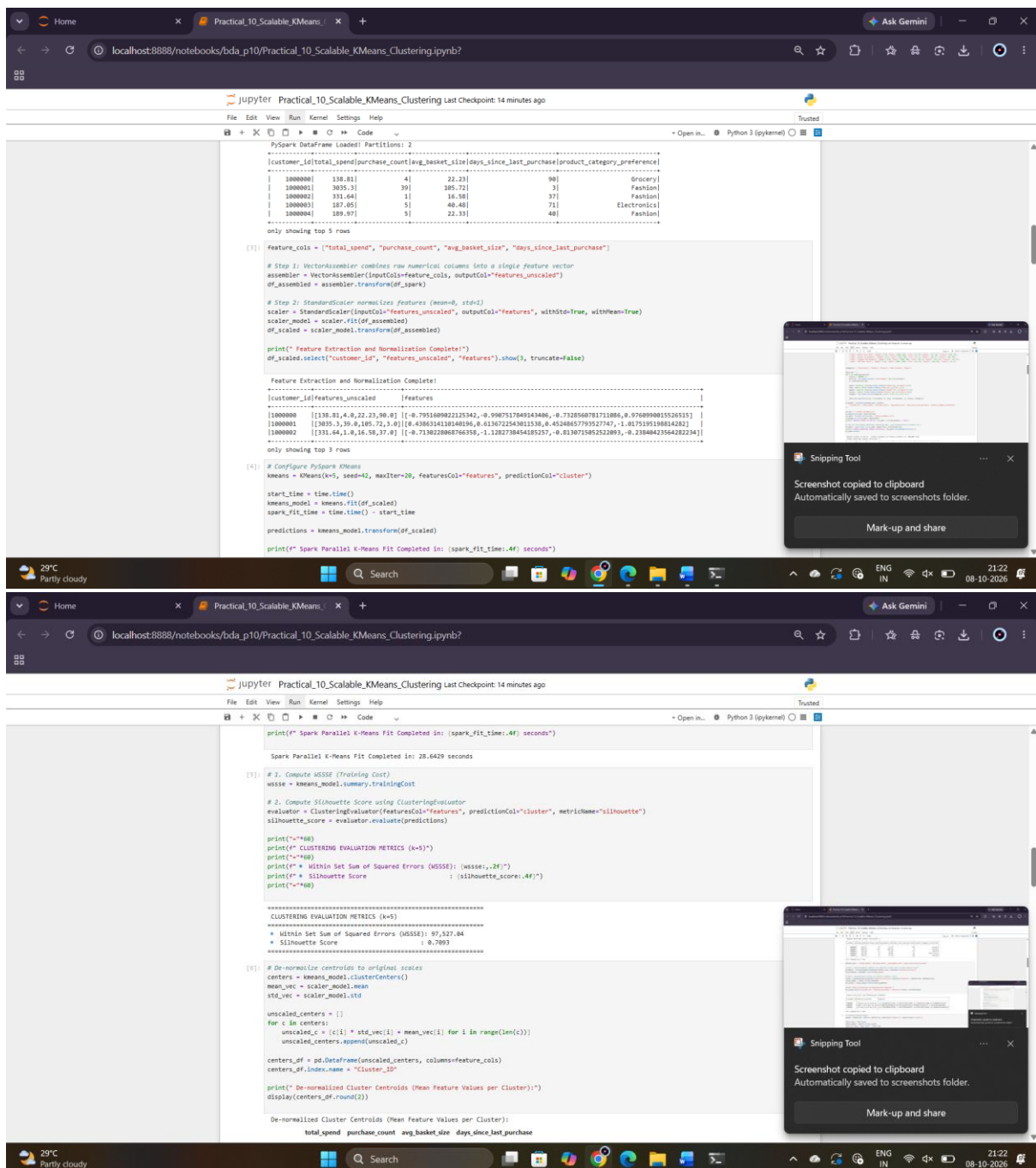
    data_rows.append((cust_id, round(spend, 2), freq, round(basket, 2), recency, category))

df_pandas = pd.DataFrame(data_rows, columns=[
    "customer_id", "total_spend", "purchase_count", "avg_basket_size", "days_since_last_purchase", "product_category_preference"
])

csv_dir = "/retail_customers_csv"
os.makedirs(csv_dir, exist_ok=True)
csv_path = os.path.join(csv_dir, "retail_customers.csv")
df_pandas.to_csv(csv_path, index=False)
print(f"Dataset written to CSV at: {csv_path} ({len(df_pandas):,} rows)")

# Load CSV into PySpark DataFrame (Simulating HDFS /user/student/retail_customers.csv)
df_spark = spark.read.csv(csv_path, header=True, inferSchema=True)
print(f"PySpark DataFrame Loaded! Partitions: {df_spark.rdd.getNumPartitions()}")
df_spark.show(5)

Dataset written to CSV at: /retail_customers_csv/retail_customers.csv (200,000 rows)
PySpark DataFrame Loaded! Partitions: 2
+-----+-----+-----+-----+-----+-----+
|customer_id|total_spend|purchase_count|avg_basket_size|days_since_last_purchase|product_category_preference|
+-----+-----+-----+-----+-----+-----+
| 1000000 | 118.81 | 4 | 22.23 | 98 | Grocery |
| 1000001 | 3035.3 | 39 | 105.72 | 3 | Fashion |
| 1000002 | 331.64 | 1 | 33.58 | 37 | Fashion |
```



The screenshot displays a Jupyter Notebook interface for a Spark-based K-Means clustering project. The notebook is titled "Practical_10_Scalable_KMeans_Clustering" and shows the following code and output:

```
PySpark DataFrame Loaded: Partitions: 2
[customer_id,total_spend,purchase_count,avg Basket Size,days_since_last_purchase,product_category_preference]
+-----+-----+-----+-----+-----+-----+
| 1000000|138.81|4|22.33|98|Grocery|
| 1000001|3035.31|39|195.72|31|Fashion|
| 1000002|331.46|1|16.58|37|Fashion|
| 1000003|187.85|5|48.48|71|Electronics|
| 1000004|189.97|5|22.33|48|Fashion|
+-----+-----+-----+-----+-----+
only showing top 5 rows

[3]: feature_cols = ["total_spend", "purchase_count", "avg_basket_size", "days_since_last_purchase"]

# Step 1: VectorAssembler combines raw numerical columns into a single feature vector
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features_unscaled")
df_assembled = assembler.transform(df_spark)

# Step 2: StandardScaler normalizes features (mean=0, std=1)
scaler = StandardScaler(inputCols="features_unscaled", outputCol="features", withStd=True, withMean=True)
scaler_model = scaler.fit(df_assembled)
df_scaled = scaler_model.transform(df_assembled)

print("Feature Extraction and Normalization Complete!")
df_scaled.select("customer_id", "features_unscaled", "features").show(5, truncate=False)

Feature Extraction and Normalization Complete!
+-----+-----+-----+
|customer_id|features_unscaled|features|
+-----+-----+-----+
|1000000|[138.81,4,22.33,98.0]|[-0.79516800022125342,-0.9907517649143486,-0.7328568781711886,0.9768999015526515]|
|1000001|[3035.3,39,195.72,31.0]|[-0.4386334118948936,-0.6136722543611518,-0.45248657793527947,-1.4817519159884282]|
|1000002|[331.46,1,16.58,37.0]|[-0.7136228865766158,-1.1282738454183257,-0.8138715851521093,-0.23860423564282243]|
+-----+-----+-----+
only showing top 3 rows

[4]: # Configure PySpark KMeans
kmeans = KMeans(k=5, seed=42, maxIter=20, featuresCol="features", predictionCol="cluster")
start_time = time.time()
kmeans_model = kmeans.fit(df_scaled)
spark_fit_time = time.time() - start_time
predictions = kmeans_model.transform(df_scaled)

print("Spark Parallel K-Means Fit Completed in: (spark_fit_time:.4f) seconds")

Spark Parallel K-Means Fit Completed in: 28.6429 seconds

[5]: # 1. Compute WSSSE (Training Cost)
wssse = kmeans_model.summary.trainingCost

# 2. Compute Silhouette Score using ClusteringEvaluator
evaluator = ClusteringEvaluator(featuresCol="features", predictionCol="cluster", metricName="silhouette")
silhouette_score = evaluator.evaluate(predictions)

print("=====")
print("CLUSTERING EVALUATION METRICS (k=5)")
print("=====")
print("Within Set Sum of Squared Errors (WSSSE): (wssse:.2f)")
print("Silhouette Score : (silhouette_score:.4f)")
print("=====")

CLUSTERING EVALUATION METRICS (k=5)
=====
Within Set Sum of Squared Errors (WSSSE): 87,527.84
Silhouette Score : 0.7893
=====

[6]: # De-normalize centroids to original scales
centers = kmeans_model.clusterCenters()
mean_vec = scaler_model.mean
std_vec = scaler_model.std

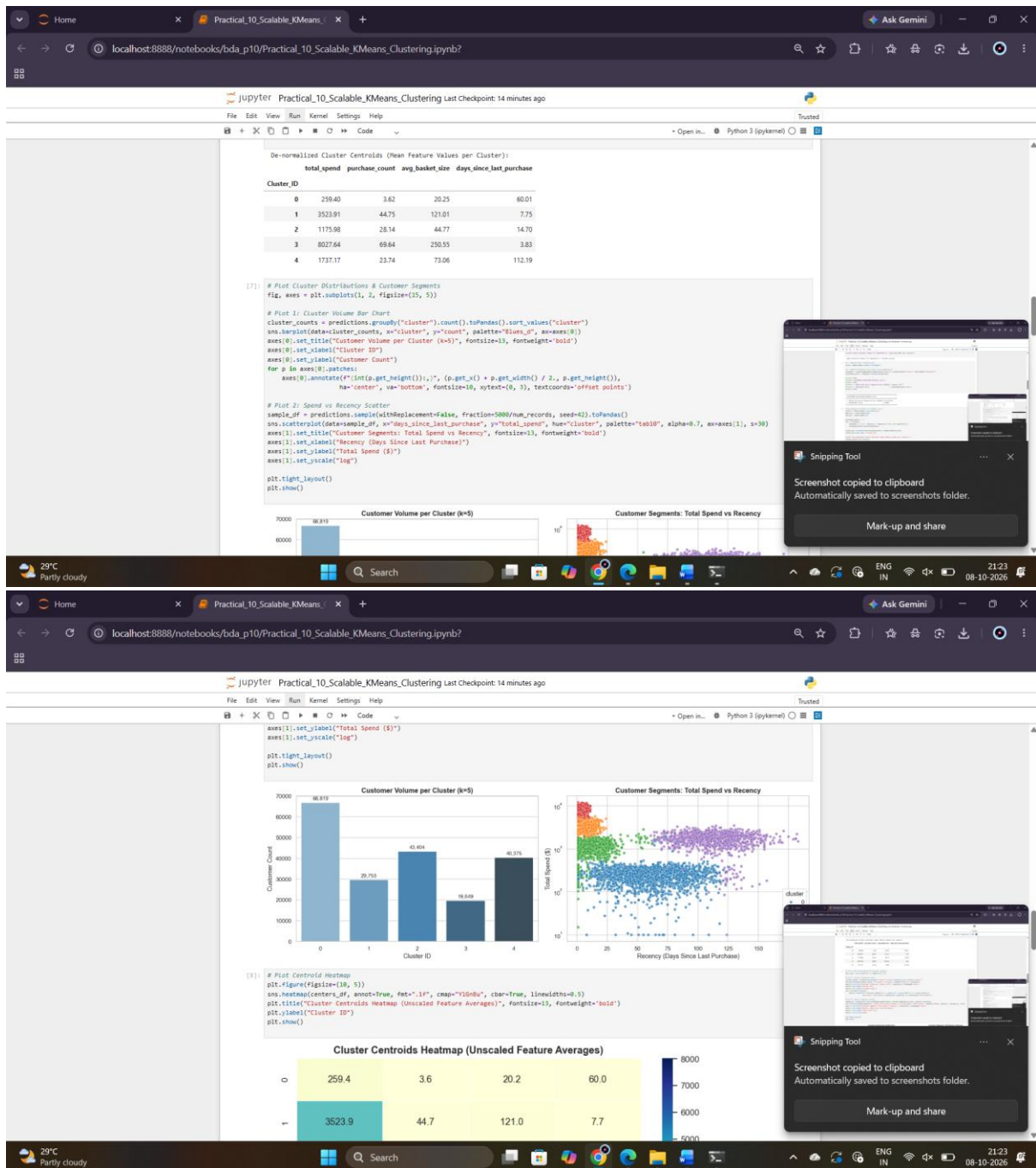
unscaled_centers = []
for c in centers:
    unscaled_c = [c[i] * std_vec[i] + mean_vec[i] for i in range(len(c))]
    unscaled_centers.append(unscaled_c)

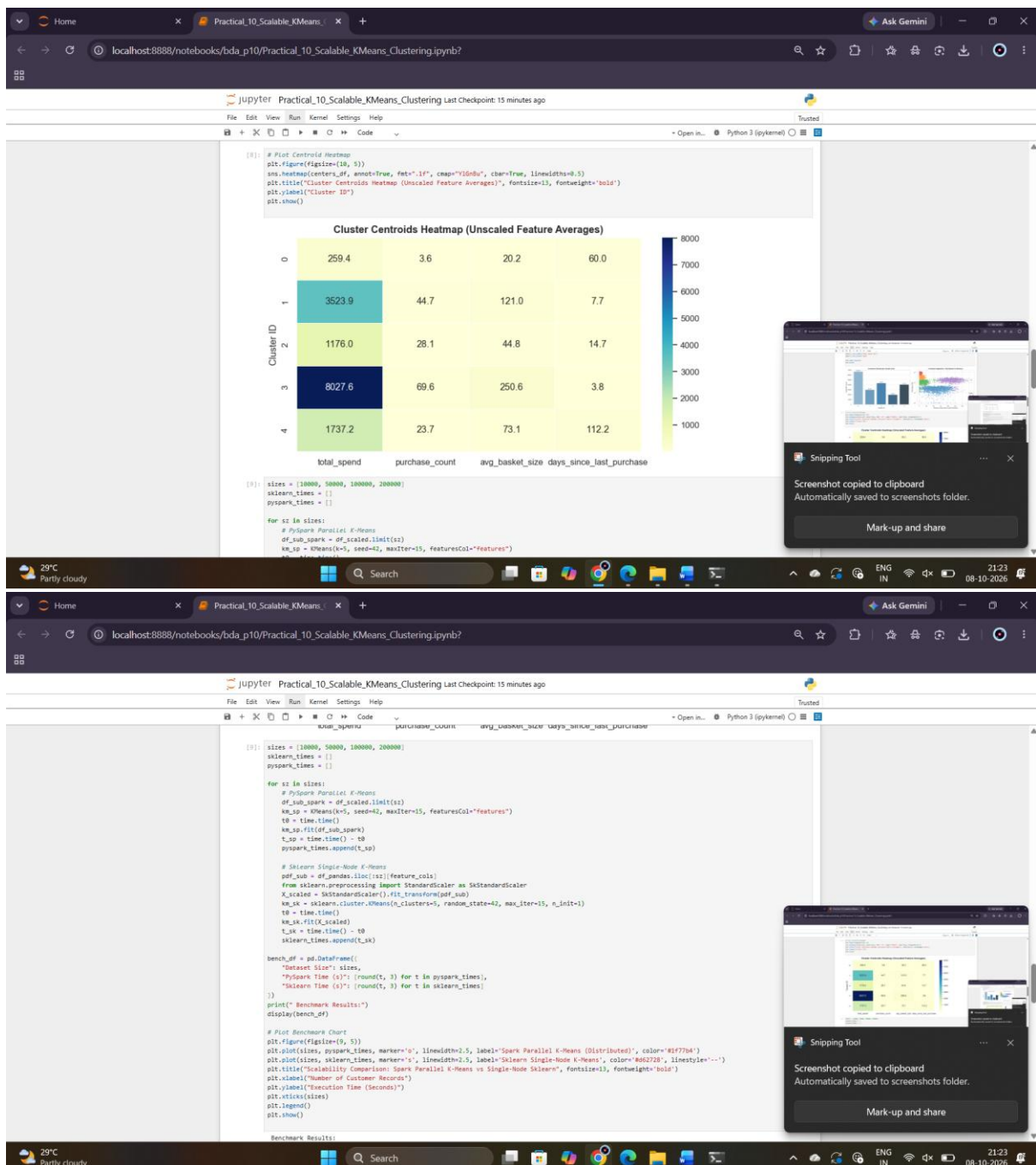
centers_df = pd.DataFrame(unscaled_centers, columns=feature_cols)
centers_df.index.name = "Cluster_ID"

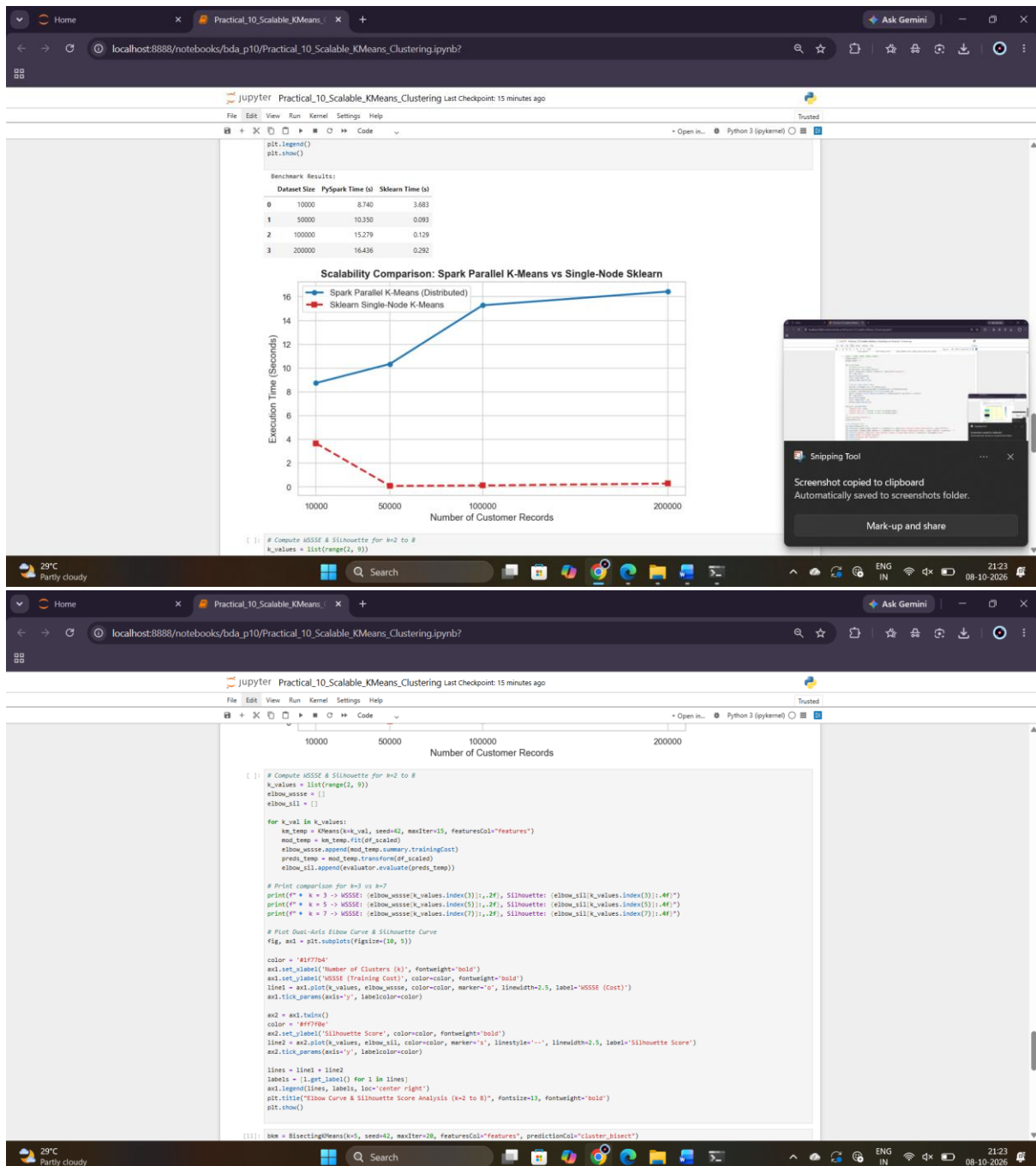
print("De-normalized Cluster Centroids (Mean Feature Values per Cluster):")
display(centers_df.round(2))

De-normalized Cluster Centroids (Mean Feature Values per Cluster):
total_spend purchase_count avg_basket_size days_since_last_purchase
```

A Snipping Tool window is visible on the right, showing a screenshot of the notebook content. The tool indicates that the screenshot was copied to the clipboard and automatically saved to the screenshots folder.







The image displays two screenshots of a Jupyter Notebook titled "Practical_10_Scalable_KMeans_Clustering". The notebook is running on a local host (localhost:8888) and is using Python 3 (ipykernel).

Top Screenshot: The notebook shows the execution of K-Means clustering. The code includes the following steps:

- Initialization of K-Means model with `k=5`, `seed=42`, and `maxIter=20`.
- Transformation of features using `df_scaled`.
- Training the model and calculating the training cost.
- Clustering evaluation using `ClusteringEvaluator` with features, prediction column, and silhouette metric.
- Comparison of Standard K-Means vs Bisecting K-Means using `pd.DataFrame`.

The output shows a comparison table:

Algorithm	WSSSE (Training Cost)	Silhouette Score	Execution Time (s)
0 Standard K-Means (Parallel)	97527.0407	0.7093	28.6429
1 Bisecting K-Means	112550.6685	0.6150	28.6489

The code continues with the preparation of features for a product category preference model, including the use of `StringIndexer`, `Indexer`, `OneHotEncoder`, and `StandardScaler`.

Bottom Screenshot: The notebook shows the execution of feature expansion. The code includes the following steps:

- Preparation of features for a product category preference model.
- Clustering evaluation using `ClusteringEvaluator` with features, prediction column, and silhouette metric.
- Feature expansion completion and silhouette score calculation.

The output shows the silhouette score with category preference: 0.6090.

Both screenshots include a "Snipping Tool" overlay indicating that the screenshot was copied to the clipboard and saved to the screenshots folder.

Key Questions / Analysis

Q1. Why do traditional K-Means fail at large scale, and how does Spark's parallel K-Means overcome this?

Traditional K-Means processes the data mainly on a single machine. When the dataset contains millions of customer records, the amount of memory and computational power required becomes very large.

A single-node implementation may suffer from:

1. High memory consumption.
2. Long execution time.
3. Limited CPU resources.
4. Difficulty processing very large datasets.
5. Possible memory overflow for extremely large datasets.

Apache Spark's MLlib provides distributed K-Means that can process data across multiple partitions and computing resources.

Spark divides the dataset into partitions and performs computations in parallel.

Spark's parallel K-Means therefore provides:

1. Distributed data processing.
2. Parallel computation.
3. Better scalability.
4. Efficient processing of large datasets.
5. Better utilization of available computing resources.

Therefore, Spark MLlib is more suitable for large-scale customer segmentation than a traditional single-node K-Means implementation.

Q2. What is the significance of the WSSSE metric? How do you use the Elbow Method to select K?

WSSSE stands for Within Set of Squared Errors.

It measures the total squared distance between each data point and the center of the cluster to which it belongs.

The WSSSE can be represented as:

$$WSSSE = \sum_{i=1}^n \|x_i - c_i\|^2$$

where:

- (x_i) represents a customer data point.
- (c_i) represents its assigned cluster center.
- (n) represents the number of customer records.

A lower WSSSE indicates that customers are closer to their respective cluster centers.

To select the appropriate value of K, the Elbow Method can be used.

The process is:

1. Run K-Means with different values of K.
2. Calculate WSSSE for each value.
3. Plot K against WSSSE.
4. Observe where the reduction in WSSSE starts becoming smaller.
5. Select the point where the curve forms an "elbow."

For example:

K WSSSE

3 Higher

4 Lower

5 Much lower

6 Slightly lower

7 Slightly lower

If the major reduction occurs up to K=5 and further increases provide only small improvements, K=5 can be selected.

Therefore, WSSSE and the Elbow Method help determine a suitable number of customer segments.

Q3. How does data partitioning in Spark affect convergence speed of K-Means?

Spark divides a large dataset into multiple partitions. These partitions can be processed in parallel by Spark's available computing resources.

Appropriate partitioning can improve K-Means performance because multiple portions of the dataset can be processed simultaneously.

If there are too few partitions:

1. Available CPU resources may not be fully utilized.
2. Parallelism may be reduced.
3. Processing may take longer.

If there are too many partitions:

1. Task scheduling overhead may increase.
2. Communication overhead can increase.
3. Performance may decrease because of unnecessary task management.

Therefore, an appropriate number of partitions helps Spark achieve better parallel processing and can reduce the time required for K-Means iterations.

Partitioning mainly affects execution performance, while the mathematical convergence of K-Means depends on factors such as initialization, feature scaling, data distribution, and the stopping criteria.

Q4. What customer behaviour pattern does each cluster represent? How can marketing strategies differ per segment?

Each K-Means cluster represents customers having similar purchasing behaviour.

The four main features used are:

1. Total Spend
2. Purchase Count
3. Average Basket Size
4. Days Since Last Purchase

Based on these characteristics, clusters can represent different customer segments.

1. High-Value Loyal Customers

These customers may have:

- High total spending.
- High purchase frequency.
- High or medium average basket size.
- Low days since last purchase.

Marketing strategy:

- Provide loyalty rewards.
- Offer premium products.
- Provide early access to new products.
- Give personalized offers.

2. Low-Value / At-Risk Customers

These customers may have:

- Low or medium spending.
- Low purchase frequency.
- High days since last purchase.

Marketing strategy:

- Send re-engagement campaigns.
- Provide discounts.
- Offer personalized recommendations.
- Send reminders and limited-time offers.

3. Frequent Buyers

These customers have relatively high purchase frequency.

Marketing strategy:

- Loyalty programs.
- Membership benefits.
- Bundle offers.
- Cross-selling and upselling.

4. High-Spending Customers

These customers generate relatively high spending even if their purchase frequency is not very high.

Marketing strategy:

- Premium offers.
- High-value product recommendations.
- Personalized campaigns.
- Exclusive promotions.

5. Occasional Customers

These customers may have moderate or low spending and purchase less frequently.

Marketing strategy:

- Promotional campaigns.
- Product recommendations.
- First/next purchase discounts.
- Campaigns designed to increase purchase frequency.

The exact meaning of each cluster should be determined from the actual cluster summary generated by the Spark program, rather than assigning labels before analyzing the results.

Q5. How does feature scaling impact the clustering result? What happens if features are not normalized?

Feature scaling is important because K-Means calculates distances between data points and cluster centers.

The customer dataset contains features with different numerical ranges.

For example:

total_spend = 15,000

purchase_count = 25

avg_basket_size = 600

days_since_last_purchase = 10

The numerical scale of total_spend is much larger than the other features.

If features are not normalized, the feature with the largest numerical range can have a greater influence on the distance calculation.

This may cause:

1. Biased cluster assignments.
2. Poor customer segmentation.
3. Less meaningful cluster centers.
4. Important smaller-scale features receiving less influence.
5. Incorrect interpretation of customer behaviour.

Therefore, StandardScaler is used to normalize the features before applying K-Means.

Feature scaling makes the features more comparable and allows K-Means to consider customer behaviour more fairly.

Supplementary Problems

1. Try $k=3$ and $k=7$ and compare WSSSE. Plot the Elbow curve.
2. Implement Bisecting K-Means (also in MLlib) and compare its clusters with standard K-Means.
3. Add a new feature: 'product category preference' and observe how cluster composition changes.

Conclusion

The Scalable K-Means Clustering on Customer Segmentation Data using Apache Spark MLlib practical was successfully completed using PySpark.

A customer transaction dataset was loaded into a Spark DataFrame and examined. The data was validated and important customer behaviour features such as total spending, purchase frequency, average basket size, and days since the last purchase were extracted.

The features were assembled into vectors and normalized using StandardScaler. Spark MLlib K-Means was then applied to divide customers into meaningful segments.

WSSSE was calculated to evaluate clustering quality, and the Elbow Method was used to compare different values of K and identify a suitable number of clusters. The cluster centers and customer characteristics were analyzed to understand different customer behaviour patterns.

The practical also demonstrated the importance of Spark data partitioning and parallel processing for large-scale machine learning. Spark K-Means was compared with single-node Scikit-learn K-Means to understand the advantages of distributed machine learning for large datasets.

The customer segments can help businesses identify high-value customers, frequent buyers, occasional customers, and at-risk customers and design targeted marketing strategies for each segment.

Overall, the practical demonstrated how Apache Spark MLlib, DataFrames, feature scaling, partitioning, K-Means clustering, WSSSE, and the Elbow Method can be used for scalable customer segmentation.